

Ejercicio 5: Las Guardias del Sultán de Brunei (Backtracking)

Enunciado

El sultán de Brunei desea colocar N guardias en un palacio de dimensiones $N \times N$ (con N^2 salas cuadradas) de manera que **ningún par de guardias se puedan ver entre sí**.

La visibilidad funciona a través de túneles rectos en la misma **fila, columna o diagonal**. La línea de visión se bloquea únicamente si hay otro guardia en medio. El objetivo es diseñar un algoritmo de **Vuelta Atrás (Backtracking)** para encontrar una colocación válida de los N guardias en las salas.

1. Análisis del Espacio de Búsqueda y Regla de Visión

Para resolver el problema tal y como lo define el sultán, debemos modelar la regla de visión de forma explícita:

- Dos salas (f_1, c_1) y (f_2, c_2) están en la misma línea de visión si comparten fila ($f_1 = f_2$), columna ($c_1 = c_2$) o diagonal ($|f_1 - f_2| = |c_1 - c_2|$).
- Un guardia en la posición A puede ver a un guardia en la posición B si están en la misma línea de visión y **no hay ningún otro guardia en el camino recto entre ellos** que obstruya los túneles.

Estrategia de Vuelta Atrás (Backtracking)

Para colocar exactamente N guardias en un tablero $N \times N$:

- Representamos el tablero como una matriz de dimensiones $N \times N$.
- Intentamos colocar los guardias de forma sistemática. Como debemos colocar N guardias en N filas y no pueden verse directamente, una estrategia eficiente es intentar colocar **exactamente un guardia por fila** (desde la fila 1 hasta la N).
- Para cada celda tentativa $(fila, col)$, lanzamos "rayos" visuales en las direcciones de los túneles para comprobar si hay algún guardia visible sin obstáculos intermedios.

2. Diseño del Algoritmo

Función de Comprobación de Visión (EsSeguro)

Para verificar si es seguro colocar un guardia en la posición (f, c) , exploramos las direcciones de los túneles (norte, sur, este, oeste y las cuatro diagonales).

En cada una de las 8 direcciones, avanzamos paso a paso desde (f, c) hacia el extremo del tablero:

- Si encontramos una celda vacía, seguimos avanzando.

- Si encontramos a otro guardia, significa que se ven directamente (puesto que es el primer obstáculo en esa línea) $\implies$ **No es seguro.**
- Si llegamos al límite del tablero sin encontrar ningún guardia, esa dirección es segura.

Algoritmo de Backtracking Principal

La función recursiva intentará colocar un guardia en la fila actual, probando cada columna de 1 a N . Si es seguro, coloca el guardia temporalmente y avanza a la siguiente fila. Si la exploración falla, deshace la asignación (vuelta atrás).

3. Algoritmo y Pseudocódigo

Función de Factibilidad (Ray-Casting con Bloqueo)

```

Función EsSeguro(tablero, N, fila, col):
    // Definimos las 8 direcciones de los túneles (df, dc)
    Direcciones = [
        (-1, 0), // Norte
        (1, 0),  // Sur
        (0, -1), // Oeste
        (0, 1),  // Este
        (-1, -1), // Noroeste
        (-1, 1), // Noreste
        (1, -1),  // Suroeste
        (1, 1)   // Sureste
    ]

    Para cada (df, dc) en Direcciones:
        paso_f = fila + df
        paso_c = col + dc

        // Avanzamos por el túnel en esa dirección paso a paso
        Mientras (1 <= paso_f <= N y 1 <= paso_c <= N):
            Si (tablero[paso_f][paso_c] == 'GUARDIA'):
                // Encontramos un guardia directamente en la línea de visión
                // (No hay ningún guardia intermedio que lo bloquee ya que es el
                Retornar Falso

            // Si la sala está vacía, el túnel sigue libre. Continuamos explorando
            paso_f = paso_f + df
            paso_c = paso_c + dc

        // Si ningún "rayo" visual ha colisionado con otro guardia, la posición es segura
        Retornar Verdadero

```

Algoritmo de Colocación Recursiva

```

Función ResolverSultan(tablero, N, fila_actual, guardias_colocados):
    // Caso Base: Hemos colocado con éxito las N guardias
    Si guardias_colocados == N:
        Retornar Verdadero

```

```

// Si hemos recorrido todo el tablero y no pusimos las N guardias, fallamos
Si fila_actual > N:
    Retornar Falso

// Intentamos colocar un guardia en la fila_actual probando cada columna
Para col desde 1 hasta N:
    Si EsSeguro(tablero, N, fila_actual, col):
        // 1. Colocar guardia en la sala (fila_actual, col)
        tablero[fila_actual][col] = 'GUARDIA'

        // 2. Intentar colocar el siguiente guardia en la siguiente fila
        Si ResolverSultan(tablero, N, fila_actual + 1, guardias_colocados + 1)
            Retornar Verdadero

        // 3. Backtracking: Deshacer la colocación si no llevó a una solución
        tablero[fila_actual][col] = 'VACÍO'

// Si no se pudo colocar ningún guardia válido en esta fila, retrocedemos
Retornar Falso

```

4. Análisis de Complejidad

- **Temporal:** En el peor de los casos, la complejidad está acotada por $O(N!)$. La función de comprobación de visibilidad tarda $O(N)$ al explorar los túneles rectos de longitud máxima N . Gracias a la poda del backtracking, la mayoría de las ramas inválidas se descartan en las primeras filas, haciendo que el algoritmo sea extremadamente rápido en la práctica.
- **Espacial:** $O(N^2)$ para representar explícitamente el palacio de $N \times N$ con sus salas, más el coste de la pila de llamadas del sistema que consume un espacio de $O(N)$ debido a la profundidad de la recursión.